

Universidad del Valle de Guatemala
Facultad de Ingeniería



INFORME DE PROYECTO 2 - MICROPROCESADORES

Integrantes

Ronald Manuel Catún Chacón – 19789

Julio Reynaldo Pellecer Morales – 241071

Hugo Carrillo - 241499

PROGRAMACIÓN DE MICROPROCESADORES

- SECCIÓN 10 -

Ciudad de Guatemala, Guatemala

2026

Tabla de contenido

Contenido

Tabla de contenido.....	2
Resumen del proyecto.....	3
Introducción.....	3
Análisis del juego seleccionado	3
Funcionamiento general	3
Elementos visuales ASCII.....	4
Interacción del usuario	4
Requisitos funcionales	4
Diseño del entorno gráfico	5
solución planteada y arquitectura concurrente.....	6
Catálogo de hilos utilizados.....	6
Catálogo de funciones principales.....	7
Mecanismos de sincronización	8
Análisis de paralelismo	8
Resultados y pruebas	9
Fortalezas y debilidades	9
Fortalezas.....	9
Debilidades y mejoras futuras.....	9
Conclusiones.....	10
Anexos	10
Estructura de archivos.....	10
Funciones/librerías técnicas utilizadas	10

Resumen del proyecto

El proyecto implementa una versión funcional de Lumines en consola. El jugador controla piezas de 2x2 bloques, las rota y las desplaza para formar cuadrados 2x2 del mismo color. Una línea de tiempo recorre el tablero y elimina las combinaciones marcadas. La meta es alcanzar 50 puntos antes de perder las 3 vidas disponibles.

El enfoque técnico usa hilos independientes para entrada del usuario, movimiento por gravedad, línea de tiempo, detección de combinaciones, bloques especiales, cálculo de puntaje y renderizado ASCII. El estado compartido se protege con mutex; la comunicación entre hilos usa semáforos, variables de condición y una barrera de arranque.

introducción

Lumines es un juego de rompecabezas en el que el jugador organiza bloques de colores para crear patrones. A diferencia de Tetris, la eliminación no ocurre de inmediato al formar una combinación: una línea de tiempo barre periódicamente el tablero y limpia los grupos válidos. Esta mecánica se presta naturalmente a una implementación concurrente porque varias actividades ocurren en paralelo: entrada del jugador, caída, detección, limpieza, puntaje y dibujo en pantalla.

Análisis del juego seleccionado

Funcionamiento general

- El tablero es una cuadrícula de 16 filas por 10 columnas.
- Cada pieza activa es un bloque 2x2 con celdas de dos colores posibles.
- El jugador mueve y rota la pieza para construir cuadrados 2x2 del mismo color.
- El hilo detector marca las combinaciones válidas y la línea de tiempo las elimina al pasar por la columna correspondiente.
- Los bloques especiales marcan regiones conectadas del mismo color, generando puntos extra.
- El jugador gana al alcanzar 50 puntos y pierde al quedarse sin vidas.

Elementos visuales ASCII

Elemento	Representacion	Descripcion
Bloque claro	OO	Celda normal de color claro.
Bloque oscuro	##	Celda normal de color oscuro.
Especial claro	@@	Bloque especial asociado al color claro.
Especial oscuro	\$\$	Bloque especial asociado al color oscuro.
Marcado	xx	Celda perteneciente a una combinación pendiente de eliminar.
Linea de tiempo	vv	Indicador superior de la columna actual.
Vacio	..	Espacio libre dentro del tablero.

Interaccion del usuario

Tecla	Accion
A / flecha izquierda	Mover la pieza una columna a la izquierda.
D / Flecha derecha	Mover la pieza una columna a la derecha.
W / Flecha arriba	Rotar la pieza 2x2 en sentido horario.
S / Flecha Abajo	Bajar la pieza una fila.
ESPACIO	Soltar la pieza inmediatamente.
P	Pausar o reanudar la partida.
Q	Terminar la partida actual.

Requisitos funcionales

- Mostrar menú principal con opciones para iniciar, ver instrucciones, consultar puntajes y salir.
- Incluir dos modalidades: modo lento y modo rápido.
- Actualizar el tablero en tiempo real desde consola usando ASCII.
- Permitir rotación, desplazamiento, caída suave y caída rápida de piezas.
- Detectar cuadrados 2x2 del mismo color y marcarlos para eliminación.
- Mover una línea de tiempo periódica que limpia combinaciones.
- Calcular puntos base, puntos por celdas eliminadas y bonos por cadenas.
- Incluir bloques especiales capaces de marcar regiones conectadas.
- Manejar victoria al llegar a 50 puntos y derrota al perder 3 vidas.
- Guardar y mostrar los mejores puntajes de ejecuciones anteriores.

Diseño del entorno grafico

El entorno visual se diseñó para una consola estándar, evitando desbordes y superposiciones. El tablero se imprime con bordes, coordenadas de fila, indicador de linea de tiempo, panel de estado y bitácora corta de eventos.

Pantalla	Contenido
Menu principal	Opciones de inicio lento/rapido, instrucciones, puntajes y salida.
Instrucciones	Objetivo, controles, mecanica de combinaciones y bloques especiales.
Puntajes	Tabla de mejores resultados persistida en high_scores.txt.
Tablero	Cuadrícula 16x10, pieza activa, linea de tiempo, vidas, nivel y puntaje.
Final	Resultado de victoria o derrota y registro del nombre del jugador.

solución planteada y arquitectura concurrente

La solución centraliza el estado del juego en una estructura GameState. El tablero, la pieza activa, el puntaje, las vidas, la cola de comandos y los eventos se consideran recursos compartidos. Cada hilo lee o modifica ese estado con protección de mutex para evitar condiciones de carrera.

La arquitectura divide responsabilidades por comportamiento del juego. Esto permite que la caída, el barrido, la entrada y el puntaje avancen sin bloquearse mutuamente. La comunicación por semáforos evita consumo de CPU innecesario en el procesamiento de comandos y de eventos de puntaje.

Catálogo de hilos utilizados

Hilo	Elemento	Comportamiento	Variables compartidas	Sincronización
inputThread	Control del jugador	Lee teclas sin bloquear la partida y las coloca en una cola.	commands, running	stateMutex, commandSem, barrera
playerThread	Jugador	Procesa movimiento, rotación, pausa, caída rápida y salida.	active, commands, board	stateMutex, commandSem
gravityThread	Caída de bloques	Mueve la pieza hacia abajo y la fija al colisionar.	active, board, lives	stateMutex
timelineThread	Línea de tiempo	Avanza por columnas, limpia celdas marcadas y reinicia cadenas.	timelineCol, board, pendingCells	stateMutex, scoreSem
detectorThread	Combinaciones	Busca cuadrados 2x2 iguales y marca celdas validas.	board, pendingSquares	stateMutex, scoreSem
specialThread	Bloques especiales	Expande marcas sobre regiones conectadas del mismo color.	board, pendingSpecialCells	stateMutex, scoreSem
scoreThread	Puntaje	Suma puntos, bonos de cadena, nivel y condicion de victoria.	score, comboInSweep, running	stateMutex, scoreSem
rendererThread	Interfaz ASCII	Redibuja el tablero, estado y eventos recientes.	board, active, events	stateMutex, condicion de redibujo

Catalogo de funciones principales

Funcion	Proposito	Relacion con hilos
spawnPiece	Genera una pieza 2x2 aleatoria y valida si puede entrar al tablero.	gravityThread
canPlace	Detecta limites, colisiones y posiciones ocupadas.	playerThread, gravityThread
landPiece	Fija la pieza activa y verifica peligro en la fila superior.	playerThread, gravityThread
detectSquaresLocked	Marca cuadrados 2x2 del mismo color.	detectorThread
expandSpecialLocked	Marca regiones conectadas al activar bloques especiales.	specialThread
clearTimelineColumnLocked	Elimina celdas marcadas en la columna actual.	timelineThread
processScoreLocked	Calcula puntos, bonos, nivel y victoria.	scoreThread
drawGameLocked	Construye la representación ASCII del tablero.	rendererThread
barrierWait	Sincroniza el arranque de todos los hilos.	Todos los hilos de juego
saveScore/loadScores	Persisten y consultan mejores puntajes.	Flujo secuencial despues de la partida

Mecanismos de sincronización

Mecanismo	Uso	Justificacion tecnica
Mutex: stateMutex	Protege tablero, pieza, puntaje, vidas, cola de comandos y eventos.	Evita condiciones de carrera cuando varios hilos modifican el mismo estado.
Semáforo: commandSem	Despierta al hilo del jugador al recibir una tecla.	Evita espera activa y separa captura de teclado del procesamiento logico.
Semáforo: scoreSem	Notifica al hilo de puntaje cuando hay combinaciones o celdas eliminadas.	Permite calcular puntos solo cuando hay trabajo pendiente.
Variable de condición	Se usa en la barrera de arranque y notificaciones de redibujo.	Coordina hilos sin bloquear indefinidamente ni depender de retardos artificiales.
Barrera de arranque	Hace que los ocho hilos inicien la partida al mismo tiempo.	Evita que un hilo avance antes de que los demás estén listos.

Análisis de paralelismo

La estimación se realizó clasificando los subsistemas por ejecución concurrente o secuencial. Los subsistemas concurrentes son entrada, jugador, gravedad, línea de tiempo, detección, especiales, puntaje y renderizado. Los subsistemas secuenciales son menú, instrucciones, carga y guardado de puntajes, inicialización y cierre.

Categoría	Componentes	Estimacion
Paralelo	8 hilos de juego activos durante la partida.	Aproximadamente 68% del comportamiento relevante.
Secuencial	Menú, persistencia, creación/destrucción de recursos y flujo posterior al juego.	Aproximadamente 32% del comportamiento relevante.
Metodo	Conteo ponderado de subsistemas y tiempo esperado durante una partida.	Estimación cualitativa respaldada por la división de responsabilidades.

Con una fracción secuencial estimada de 0.32, la Ley de Amdahl sugiere un límite teórico cercano a 2.94x en cuatro núcleos: $1 / (0.32 + 0.68/4)$. En la práctica, el rendimiento está limitado por la consola y por la sincronización necesaria para mantener coherente el tablero.

Resultados y pruebas

Comandos usados para validar la entrega en Windows con MinGW/TDM-GCC:

mingw32-make clean

mingw32-make

mingw32-make self-test

SELF_TEST_OK: detección, puntaje, bloque especial y línea de tiempo funcionan.

Prueba	Resultado esperado	Estado
Compilación	Genera lumines.exe sin errores.	Aprobada
Self-test	Valida detección 2x2, puntaje, especial y línea de tiempo.	Aprobada
Menu	Permite iniciar, ver instrucciones, consultar puntajes y salir.	Implementada
Modo lento/rapido	Cambia velocidad de caída y línea de tiempo.	Implementada
Victoria/derrota	Victoria a 50 puntos; derrota con 3 vidas perdidas.	Implementada
Puntajes	Guarda los mejores resultados en high_scores.txt.	Implementada

Fortalezas y debilidades

Fortalezas

- La implementación separa claramente las responsabilidades por hilo.
- Usa varios mecanismos de sincronización exigidos por la rúbrica.
- No depende de librerías gráficas externas, por lo que es fácil de compilar.
- Incluye prueba automática para demostrar la lógica principal.
- El sistema de puntaje contempla cuadrados, celdas limpiadas, cadenas y especiales.

Debilidades y mejoras futuras

- La representación ASCII simplifica los colores reales de Lumines a dos símbolos principales.
- La física de limpieza es una aproximación educativa y no una réplica exacta cuadro por cuadro del juego comercial.
- podrían agregarse más niveles, música, temporizadores visuales y configuración avanzada de dificultad.

- La tabla de puntajes es local; una versión futura podría almacenar perfiles o estadísticas por jugador.

Conclusiones

El proyecto demuestra que un juego de rompecabezas puede modelarse como un sistema concurrente cuando sus eventos se separan en procesos independientes. El uso de Pthreads obliga a coordinar acceso al tablero, puntajes y comandos, por lo que los mecanismos de sincronización son parte esencial del diseño y no solo un requisito agregado.

La solución cumple con las funciones principales solicitadas para Lumines: modos lento y rápido, línea de tiempo, rotación, desplazamiento, bloques especiales, puntajes, cadenas, victoria, derrota y reinicio mediante menú. También deja una base escalable para nuevas piezas, mas colores y niveles de dificultad progresivos.

Anexos

Estructura de archivos

```
LuminesPthreads/
  Makefile
  README.md
  src/main.cpp
  tests/PLAN_DE_PRUEBAS.md
  high_scores.txt (generado al ejecutar)
```

Funciones/librerías técnicas utilizadas

Librería	Funcion/recurso	Uso en el programa
pthread.h	pthread_create, pthread_join	Creacion y cierre de hilos POSIX.
pthread.h	pthread_mutex_t	Proteccion del estado compartido.
pthread.h	pthread_cond_t	Barrera de arranque y senales de redibujo.
semaphore.h	sem_t, sem_wait, sem_post	Colas de comandos y eventos de puntaje.
conio.h / termios.h	_kbhit/_getch o modo no canonico	Lectura de teclado en tiempo real.
iostream/fstream	cout, ifstream, ofstream	Interfaz de consola y puntajes persistentes.
random	mt19937	Generacion de colores y bloques especiales.